

Parallelizing Local Histogram Equalization: An OpenMP and CUDA Comparative Analysis

Dipesh Kant (2025MCS2110)

March 27, 2026

Abstract

This report details the parallel implementation and optimization of Local Histogram Equalization (LHE) for 3D point clouds. Given the computational complexity of exact spatial neighborhood queries, we explore three algorithmic pipelines: Exact K-Nearest Neighbors (KNN), Approximate KNN using spatial hashing, and K-Means clustering. We present a comparative performance analysis between a multi-core CPU implementation utilizing OpenMP and a massively parallel GPU implementation using CUDA. Through advanced hardware-level optimizations, including shared memory tiling, dynamic block reductions, targeted loop unrolling, and register bypassing, the CUDA K-Means implementation achieved a peak execution time of 0.149 seconds, representing a 6.4x speedup over the OpenMP baseline.

1 Introduction

Local Histogram Equalization (LHE) is a critical technique for enhancing the contrast of 3D point cloud datasets. Unlike global equalization, LHE relies on the spatial neighborhood of every individual point to compute a localized Cumulative Distribution Function (CDF). As the size of point clouds grows, the computational cost of finding local neighbors increases exponentially. This project leverages parallel computing architectures to accelerate this process, exploring both exact and approximate spatial querying techniques to balance accuracy and execution time.

2 Problem Statement

Given a 3D point cloud of $N = 100,000$ points, where each point has coordinates (X, Y, Z) and an intensity $I \in [0, 255]$, the primary objective is to map the intensity of each point based on the local CDF of its $K = 128$ nearest neighbors. For iterative spatial grouping approaches, a convergence cutoff of $t = 50$ maximum iterations is enforced.

The primary bottleneck is the spatial search. A naive Exact KNN requires computing $O(N^2)$ distances, resulting in 10 billion operations. To address these memory and computational bottlenecks, we implement K-Means clustering and a grid-based Approximate KNN, optimizing all three kernels across both OpenMP (CPU) and CUDA (GPU) platforms.

3 Algorithms and Implementation

To overcome the computational expense of local spatial searches, we implemented three distinct pipelines, balancing theoretical accuracy with hardware-aware optimization.

3.1 Exact K-Nearest Neighbors (KNN)

The Exact KNN computes all $O(N^2)$ distances to guarantee a mathematically perfect local CDF.

- **Local Max-Heap:** Each GPU thread evaluates a single target point against the entire dataset. It maintains the $K = 128$ closest neighbors in a thread-local max-heap array to avoid slow dynamic memory allocation.
- **Deterministic Tie-Breaking:** To guarantee zero Mean Absolute Error (MAE) regardless of warp execution order, we implemented a consistent lexicographical tie-breaker. Equal distances are resolved by comparing X , then Y , then Z coordinates, and finally the original array index.
- **Fused Mapping:** Once the heap is populated, the thread calculates the local CDF and applies the histogram equalization formula in the same kernel pass.

3.2 Approximate KNN via Spatial Hashing

To bypass the $O(N^2)$ bottleneck, this hybrid CPU-GPU approach partitions the space into a grid, pruning the search area to local voxels.

- **Voxelization & Hashing (CPU/OpenMP):** We use OpenMP parallel reductions to find the global bounding box. Based on the K/N ratio, we define an optimal voxel size. Each point is assigned a 3D coordinate and hashed into a 1D index using a prime-modulo function.
- **Memory Linearization (CPU):** Points are sorted using `std::sort` by their hash ID. We build `start_ptr` and `end_ptr` arrays to map hash IDs to contiguous memory blocks.
- **Bounded Search (CUDA):** Instead of querying the full dataset, GPU threads only evaluate points residing within their immediate $3 \times 3 \times 3$ neighboring voxels, executing the max-heap logic on a substantially reduced subset of points.

3.3 K-Means Clustering

This approach groups points into $K = 128$ spatial clusters, allowing points within the same cluster to share a localized histogram, significantly reducing redundant calculations.

- **Centroid Assignment:** Centroids are initialized to the first K points. A CUDA kernel assigns points to the nearest cluster using the deterministic tie-breaking logic.
- **Shared Memory Reduction:** To update centroids without excessive global atomic contention, threads first aggregate X, Y, Z coordinates and point counts in block-level `__shared__` memory. Only the summarized block-level data is atomically added to global memory.
- **Segmented Equalization:** After convergence (or $t = 50$ iterations), three lightweight kernels sequentially build the histograms for each cluster, compute the CDFs, and apply the final mapped intensities back to the point cloud.

4 Parallelization & Hardware Optimizations

4.1 OpenMP (CPU) Optimizations

The baseline parallelization was achieved using OpenMP on a multi-core CPU architecture.

- **Thread-Level Parallelism:** The primary outer loops for point evaluation were parallelized using `#pragma omp parallel for`.
- **Data Reductions:** Bounding box calculations for the Approximate KNN were optimized using OpenMP reduction clauses (`reduction(min:...)` `reduction(max:...)`) to prevent race conditions without the overhead of critical sections.

4.2 CUDA (GPU) Optimizations

To approach the physical limits of the Tesla K40 GPU, several architectural optimizations were applied:

- **Shared Memory Tiling (Exact KNN):** Global memory reads are a major bottleneck. We implemented shared memory tiling, loading blocks of 128 points into fast `__shared__` memory to significantly reduce global VRAM transactions.
- **Fast Register Bypass (Exact KNN):** Maintaining the 128-element local max-heap causes L1 cache thrashing. We optimized this by dedicating a persistent hardware register to hold the max-heap root value, bypassing the heap array entirely for the vast majority of distance checks.
- **Dynamic Block Reductions (K-Means):** To optimize the centroid update phase, we utilized dynamic shared memory to perform block-level reductions. Each block aggregates its local sums and counts in fast `__shared__` memory, pushing only final block-level aggregates to global memory to significantly reduce atomic collisions.
- **Targeted Loop Unrolling:** To hide instruction latency and pipeline hardware memory reads, explicit `#pragma unroll` directives were applied at critical bottlenecks:
 - *Exact KNN:* The inner shared-memory tile evaluation loop was partially unrolled (`#pragma unroll 16`), and the 128-element heap initialization and reduction loops were completely unrolled to eliminate jump-instruction overhead.
 - *Approximate KNN:* The nested $3 \times 3 \times 3$ spatial grid search loops were fully unrolled to flatten the execution graph and minimize branch prediction stalling.
- **CPU Data Reordering (Approx. KNN):** To prevent "pointer chasing," we executed a `std::sort` on the CPU prior to kernel launch, sorting points by their spatial hash ID. This guarantees contiguous VRAM access and perfect memory coalescing.

4.3 Compilation Strategy & Makefile Configuration

To ensure the codebase fully leverages the underlying hardware, the project is compiled using NVIDIA's `nvcc` compiler with specific optimization flags defined in the `Makefile`:

- `-O3`: Enables maximum compile-time optimization, allowing the compiler to perform advanced loop vectorization and function inlining.
- `-arch=sm_35`: Explicitly targets the compute capability 3.5 architecture of the Tesla K40 GPU. This ensures the generated PTX assembly is properly optimized for Kepler's specific register limits and memory hierarchy.
- `-Xcompiler -fopenmp`: Instructs `nvcc` to pass the OpenMP flag to the host CPU compiler (e.g., GCC). This is required to enable the multi-threading directives used in the CPU-side voxelization and bounding-box reduction phases.

5 Performance Evaluation

The algorithms were tested on the HPC cluster using a dataset of $N = 100,000$ points, $K = 128$, and $t = 50$. The strategic memory, instruction-level, and register optimizations on the GPU yielded significant reductions in execution time compared to the OpenMP baseline.

Table 1: Execution Times and CUDA Speedup ($N = 100,000$)

Algorithm	OpenMP Time (s)	CUDA Time (s)	Speedup (S)
Exact KNN	5.349	2.675	2.00x
Approximate KNN	1.028	0.486	2.11x
K-Means Clustering	0.965	0.149	6.47x

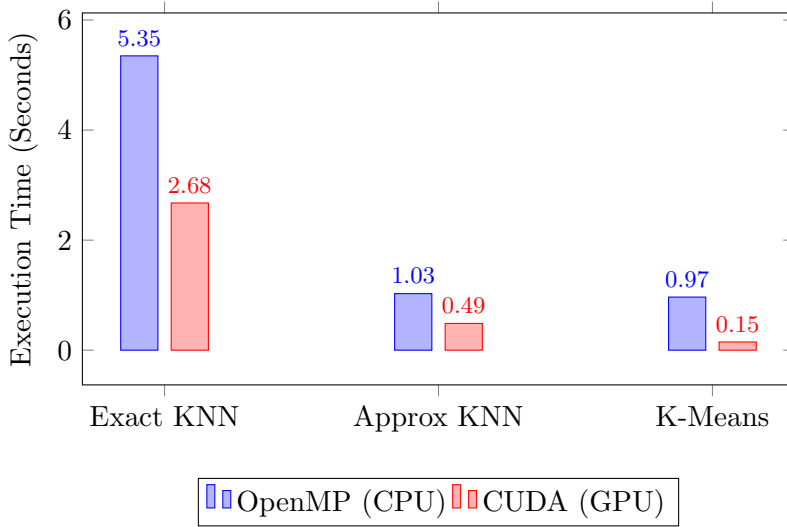


Figure 1: Performance comparison between OpenMP and CUDA implementations.

The Approximate KNN successfully reduced the $O(N^2)$ bottleneck down to sub-second execution via coalesced voxel searches. K-Means proved to be the fastest approach, scaling exceptionally well on the GPU due to localized shared memory block reductions, which effectively mitigated global memory bottlenecks.

6 Conclusion

While OpenMP provides a straightforward path to multi-core acceleration, the massively parallel architecture of the GPU, when carefully optimized for memory coalescing, cache hierarchy, and instruction pipelining, provides vastly superior performance for spatial neighborhood problems. The 6.47x speedup achieved in the K-Means implementation highlights the necessity of architectural awareness in High-Performance Computing.